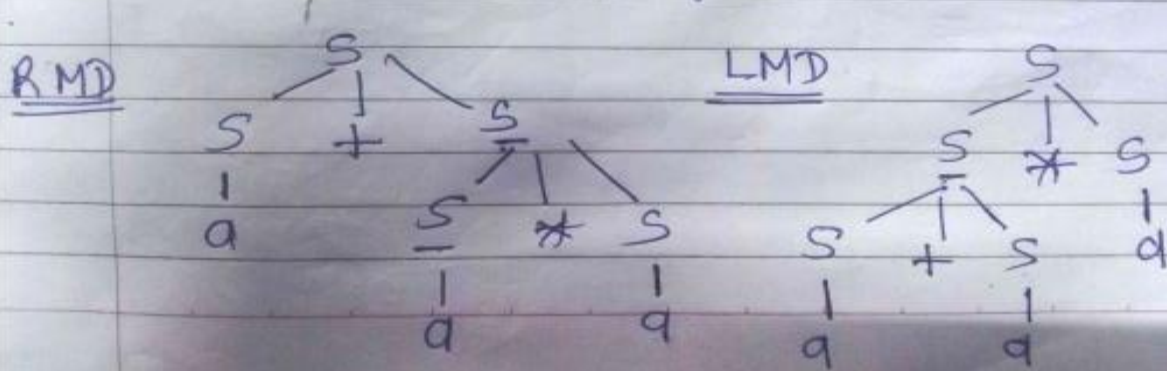


TCS (Sem-V) Model Answer May 2023

a1) The grammar given is
 $S \rightarrow s + s / s - s / s * s / s / s / (s) / \epsilon$
 generate the following string using
 parse tree
 $a + a * a$



Both the trees are different hence
 the given grammar is ambiguous.

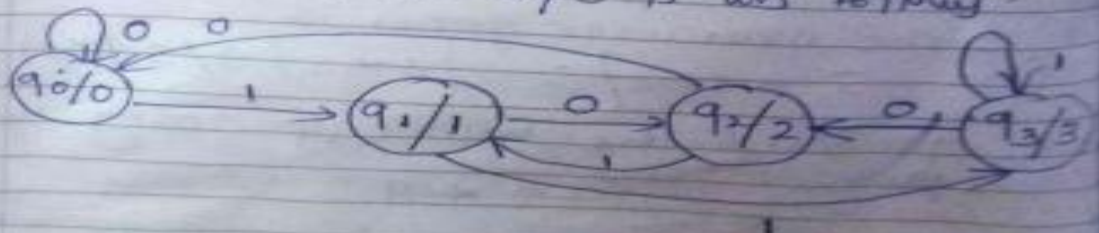
Q1) b. Moore m/c to remainder modulo
 4 for binary number.

$\Sigma = \{0, 1\}$, $\Delta = \{0, 1, 2, 3\}$

Transition table is as follows.

	0	1	Δ
q_0	q_0	q_1	0
q_1	q_2	q_3	1
q_2	q_0	q_1	2
q_3	q_2	q_3	3

Hence the m/c is as follows.



Difference between NPDA and DPDA:

S. No	DPDA(Deterministic Pushdown Automata)	NPDA(Non-deterministic Pushdown Automata)
1.	It is less powerful than NPDA. <u>Example:</u> We can only construct DPDA for odd-length palindromes and not for even length palindromes.	It is more powerful than DPDA. <u>Example:</u> NPDA can be constructed for both even-length and odd-length palindromes.
2.	It is possible to convert every DPDA to a corresponding NPDA.	It is not possible to convert every NPDA to a corresponding DPDA.
3.	The language accepted by DPDA is a subset of the language accepted by NPDA.	The language accepted by NPDA is not a subset of the language accepted by DPDA.
4.	The language accepted by DPDA is called DCFL(Deterministic Context-free Language) which is a subset of NCFL(Non-deterministic Context-free Language) accepted by NPDA.	The language accepted by NPDA is called NCFL(Non-deterministic Context-free Language).

Q1. D)

Chomsky Hierarchy represents the class of languages that are accepted by the different machines.

Chomsky hierarchy

Hierarchy of grammars according to Chomsky is explained below as per the grammar types –

Type 0 – It is an Unrestricted grammar

Unrestricted grammar – an **unrestricted grammar** is a 4-tuple (T, N, P, S) , which consisting of –

T = set of terminals

N = set of nonterminal

P = as set of productions, of the form –

$v \rightarrow w$

where v and w are strings consisting of nonterminal and terminals.

S = is called the **start symbol**.

Example – Turing Machine (TM)**Type 1 – Context-sensitive grammars**

All productions are of form,

$v \rightarrow w$ where $|v| < |w|$

$uAv \rightarrow uwv$ with $w \neq \epsilon$,

i.e., $A \rightarrow w$ but only in the context of $u _ v$.

A context-sensitive grammar is equivalent to a **linear bounded** and **context-sensitive language**.

Example – Linear Bounded Automaton (LBA)**Type 3 – Context-free grammars –**

All productions are of the form –

$A \rightarrow x$ – where A is nonterminal, x is a string of nonterminal and terminals, A context-free grammar is equivalent to a **pushdown automaton (PDA)** and to **context free languages**.

Example – Pushdown Automaton (PDA)**Type 3 – Regular grammars**

All productions are of the form –

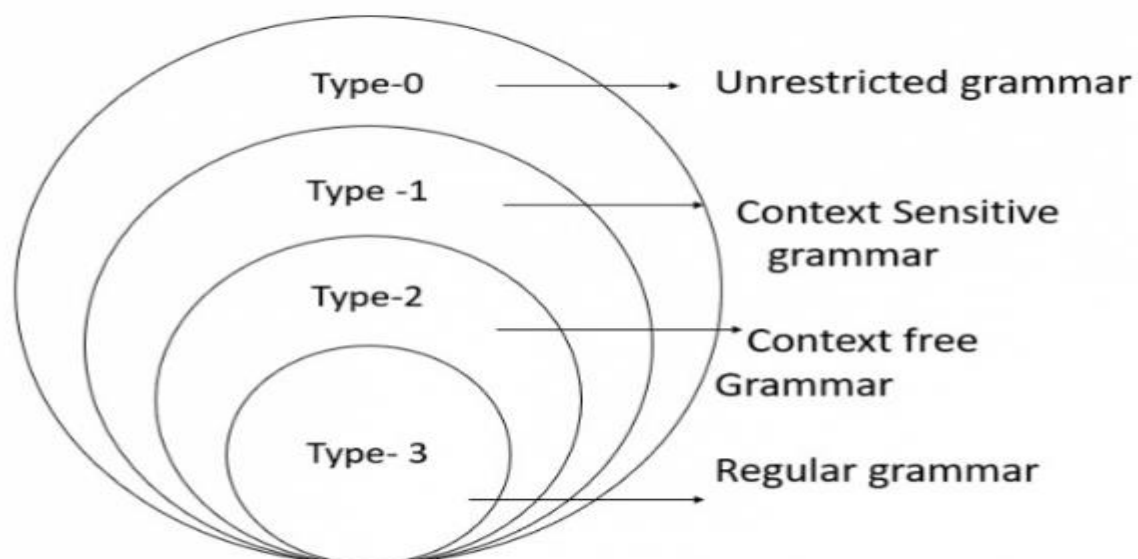
$A \rightarrow xB$

$A \rightarrow x$ where A, B are nonterminal, x is a string of terminals.

Regular grammars are equivalent to **regular sets** and to **finite automata**.

Grammar type	Grammar accepted	Language accepted	Automaton
Type 0	unrestricted grammar	recursively enumerable language	Turing Machine
Type 1	context-sensitive grammar	context-sensitive language	linear-bounded automata
Type 2	Context-free grammar	Context-free language	Push down automata
Type 3	regular grammar	regular language	finite state automaton

The Chomsky hierarchy is depicted in the diagram given below –



Steps to convert CFG to CNF

Step-1 Convert the grammar in simplified form by removing Null Production, Unit Productions and Useless Symbols.

Step-2

Convert to Binary Productions:

- Ensure that every production with more than two non-terminals on the right side is converted to a series of binary productions (rules with exactly two non-terminals on the right side).
- For a production $A \rightarrow BCD$, introduce new non-terminals (e.g., X) so that you can rewrite it as:

$$A \rightarrow BX, \quad X \rightarrow CD$$

- Repeat until all rules are binary.

Step-3

Ensure Terminal Rules Only Have One Terminal:

- For any production of the form $A \rightarrow aB$ or $A \rightarrow Ba$ (where a is a terminal and B is a non-terminal), introduce a new non-terminal (e.g., X) so that:

$$A \rightarrow XB \quad \text{and} \quad X \rightarrow a$$

- This step ensures that each terminal is generated by a rule in the form $A \rightarrow a$.

Step-4

Finalize CNF Form:

- After applying the above transformations, all productions should now follow the CNF structure:
 - $A \rightarrow BC$ (for two non-terminals),
 - $A \rightarrow a$ (for a terminal),
 - and optionally $S \rightarrow \epsilon$ if ϵ is part of the language.

$$\begin{aligned} S &\rightarrow ASB / a / bb \\ A &\rightarrow aSA / a \\ B &\rightarrow SbS / bb \end{aligned}$$

CNF is the form in which the production contains either two non-terminals or a single terminal.

The first production can be written as

$$\begin{aligned} S &\rightarrow AsB / a / BbBb \\ Bb &\rightarrow b \\ As &\rightarrow AS \end{aligned}$$

In the second production of A, we can write,

$$\begin{aligned} A &\rightarrow ASA \quad (\text{a is replaced with A}) \\ A &\rightarrow a \quad \Rightarrow \quad \begin{aligned} A &\rightarrow AsA \\ As &\rightarrow AS \\ A &\rightarrow a \end{aligned} \end{aligned}$$

In the third production of B,

$$\begin{aligned} B &\rightarrow SBbS & (Bb \rightarrow b) \\ Sb &\rightarrow SBb & \Rightarrow \quad \begin{aligned} B &\rightarrow SbS \\ Sb &\rightarrow SBb \end{aligned} \end{aligned}$$

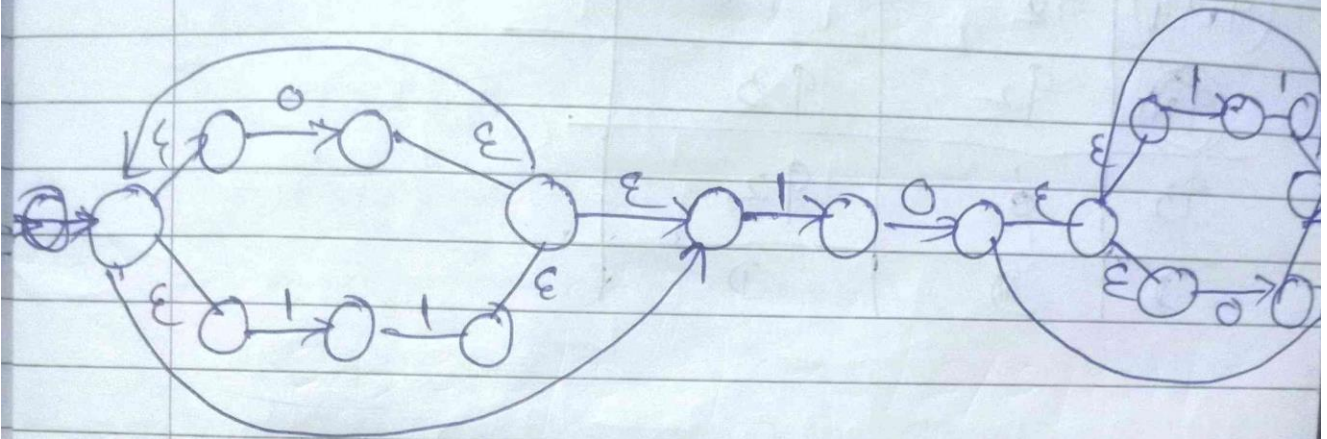
So final production are

$$\begin{aligned} S &\rightarrow AsB / a / BbBb \\ As &\rightarrow AS \\ Bb &\rightarrow b \\ A &\rightarrow AsA \\ As &\rightarrow AS \\ A &\rightarrow a \\ B &\rightarrow SbS \\ Sb &\rightarrow SBb \\ B &\rightarrow BbBb \end{aligned}$$

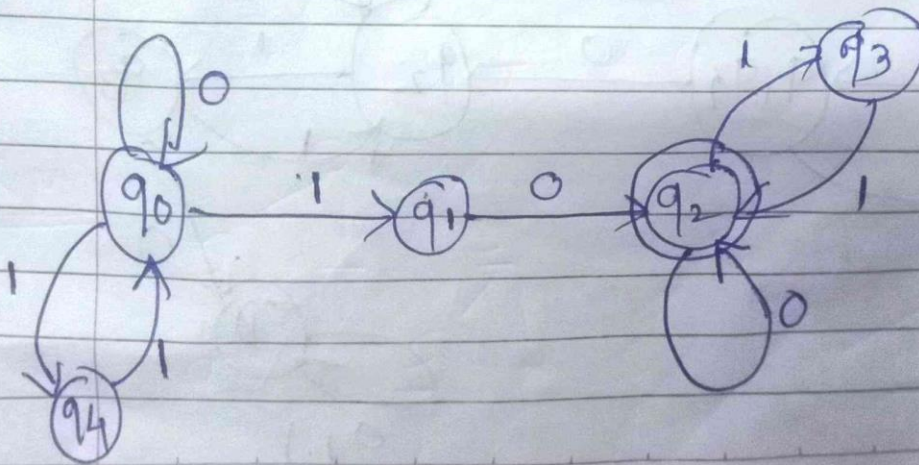
Q2b) RE $\rightarrow$ NFA- ϵ and convert to DFA

$(0+11)^*(10)(11+0)^*$

ϵ -NFA is as follows



So NFA as follows

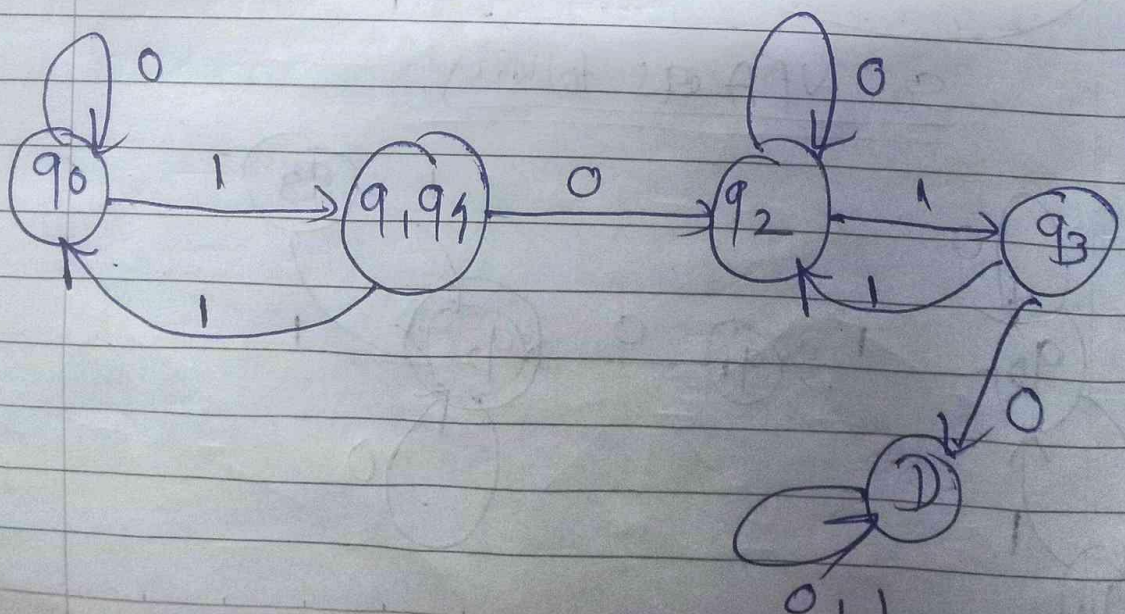


Transition table of NFA is followy

	0	1
q_0	q_0	(q_1, q_4)
q_1	q_2	ϕ
q_2	q_2	q_3
q_3	ϕ	q_2
q_4	ϕ	q_0

Now we will
convert this
NFA to MDFA
as followy

	0	1
q_0	q_0	(q_1, q_4)
(q_1, q_4)	q_2	q_0
q_2	q_2	q_3
q_3	ϕ	q_2
ϕ	ϕ	ϕ



Q. 3) A

Q3A) PDA for $L = \{a^n b^m c^n \mid m, n \geq 1\}$

take a sample string of given language.

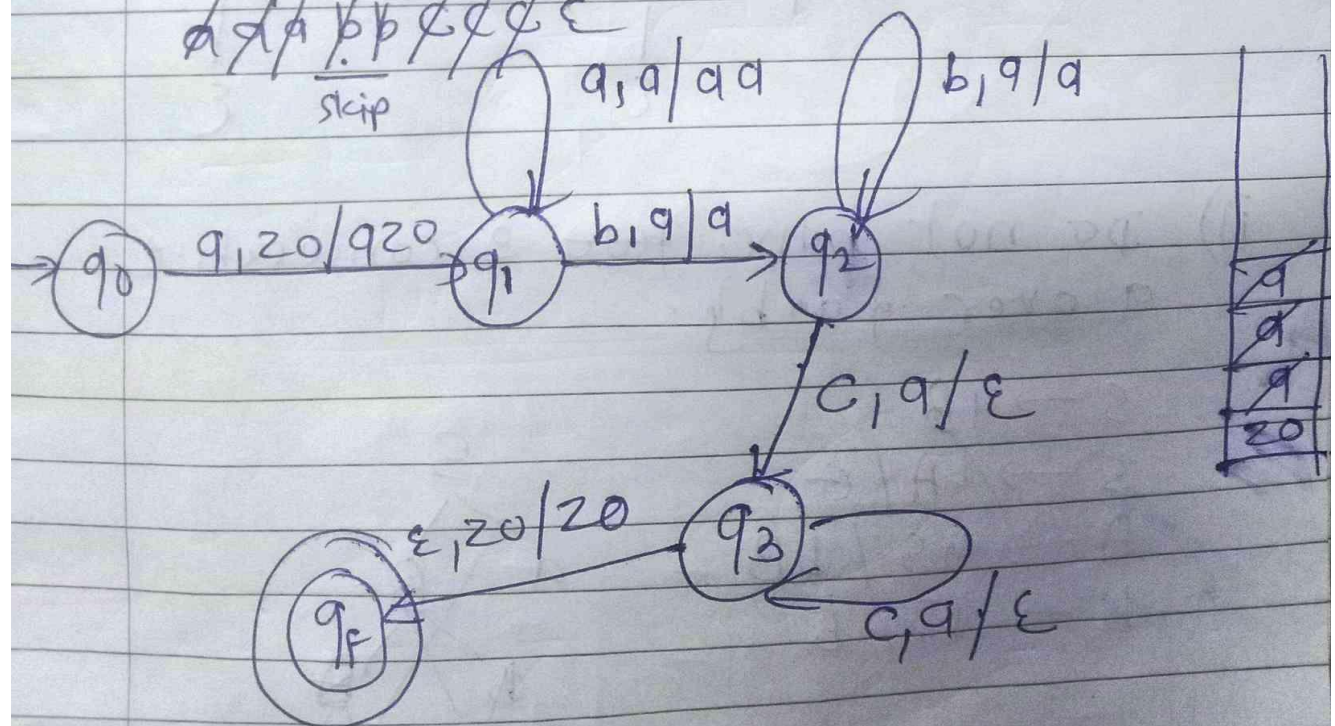
aaabbbccc/ε/ε

→ our processing in PDA is like

- 1) push all the a's
- 2) skip for b's in the string.
- 3) pop all the a's, for every occurrence of c,

→ if the string is complete and stack is empty, the PDA is working correctly.

~~a~~/~~a~~/~~a~~ ~~b~~/~~b~~/~~b~~ ~~c~~/~~c~~/~~c~~ ε



Q. 3) B

Pumping Lemma for RL

- It gives a method for pumping (generating) many substrings from a given string.
- In other words, we say it provides means to break a given long input string into several substrings.
- It gives necessary condition(s) to prove a set of strings is not regular.

Theorem

For any regular language L , there exists an integer P , such that for all w in L

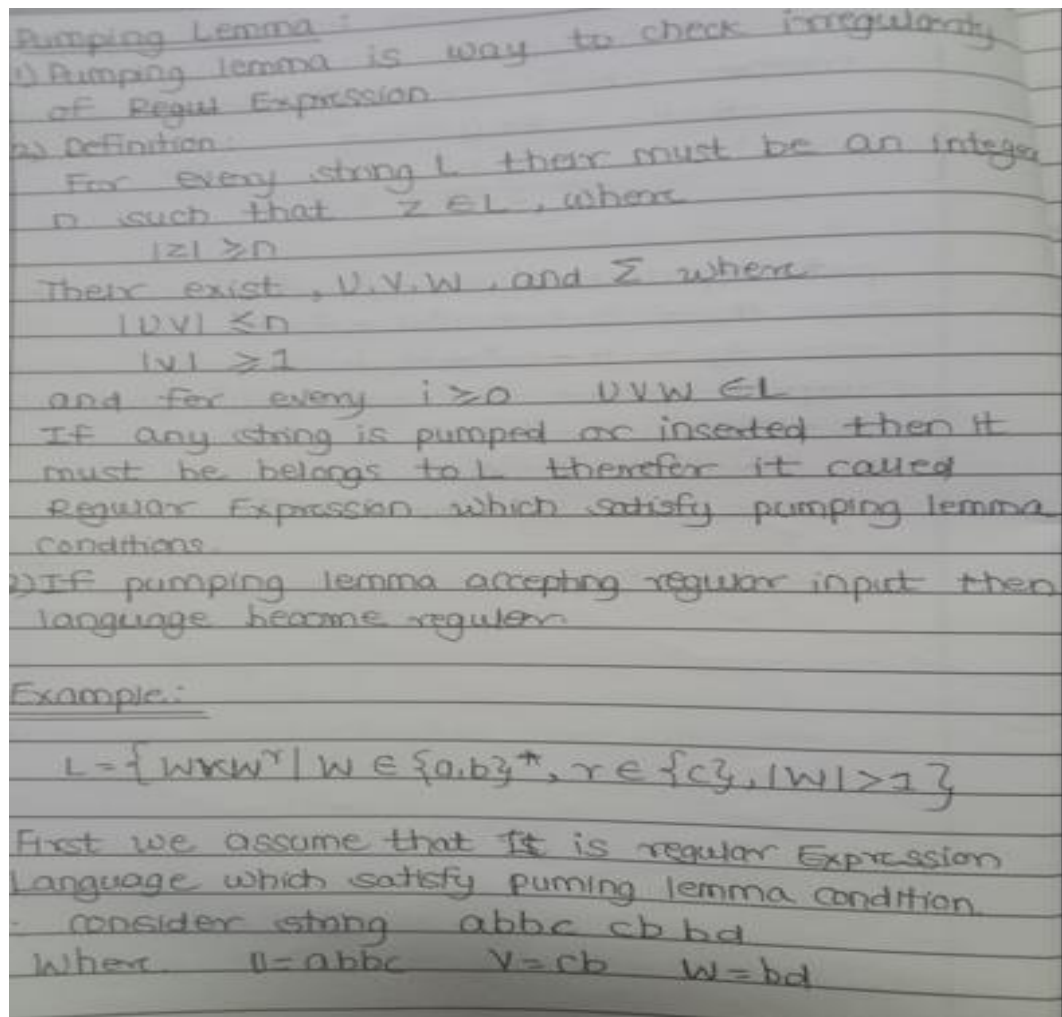
$|w| \geq P$

We can break w into three strings, $w=xyz$ such that.

(1) $|x| < P$

(2) $|y| > 1$

(3) for all $k \geq 0$: the string xy^kz is also in L



1) Do not have 3 consecutive a's nor {a, b}

$$S \rightarrow aA / bS / \epsilon$$

$$A \rightarrow aB / bS / \epsilon$$

$$B \rightarrow bS / \epsilon$$

ii) Language having equal no of 0 and 1

$$S \rightarrow 0S1 / 1S0 / 01$$

Q. 4) B

Applications of FA, PDA, TM

1. **Finite Automata** A finite automaton is a mathematical model consisting of states, input symbols, and a transition function. It reads input symbols and transitions between states based on the transition function, either accepting or rejecting the input. It's used to model and analyze systems with a finite number of states in various fields. Some applications of finite automata are as follows:
 1. **Recognizing patterns in text or other data:** Finite automata can be used to search for and recognize specific patterns in text or other data. For example, a finite automaton can be used to recognize email addresses, URLs, or phone numbers in text. This is commonly used in text processing software; such as search engines or spam filters.
 2. **Implementing regular expressions in programming languages:** Regular expressions are a way to describe patterns in text using a set of rules. These rules can be translated into a finite automaton, which is used to match the pattern in the input text. Many programming languages, such as Perl, Python, and Java, use finite automata to implement regular expressions.

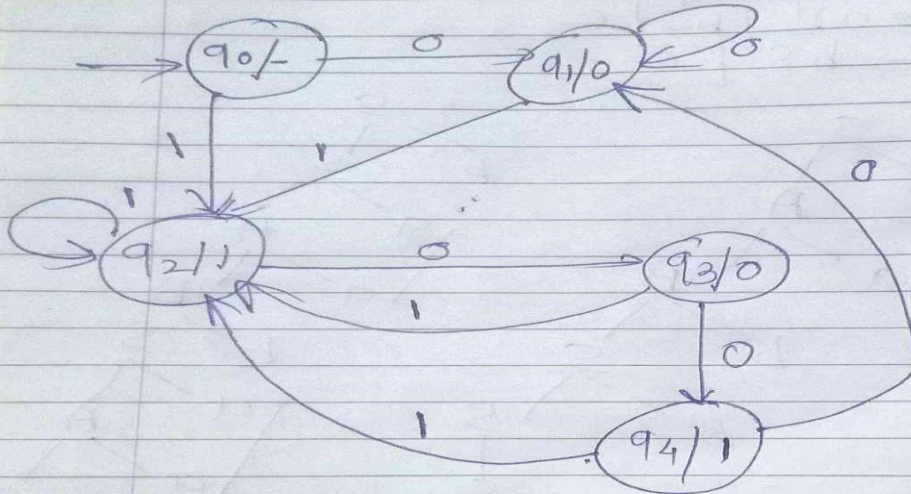
3. **Validating input in form fields on websites:** Finite automata can be used to validate user input in form fields on websites. For example, a finite automaton can be used to ensure that a user enters a valid email address or password that meets certain criteria, such as length or character requirements.
 4. **Modeling digital circuits and switching systems:** Finite automata can be used to model digital circuits and switching systems, such as those found in computers and other electronic devices. This can help engineers design and optimize these systems for maximum efficiency and performance.
 5. **Designing and analyzing computer algorithms and programs:** Finite automata can be used to design and analyze computer algorithms and programs. They can help programmers understand the behavior of the program and identify potential errors or inefficiencies.
 6. **Building compilers and interpreters:** Finite automata are used in the development of compilers and interpreters, which are programs that translate high-level programming languages into machine code that can be executed by a computer. Finite automata are used to recognize and process the syntax and semantics of the programming language.
 7. **Analyzing the behavior of software and hardware systems:** Finite automata can be used to analyze the behavior of software and hardware systems, such as operating systems or network protocols. They can help identify bugs or vulnerabilities in the system and optimize its performance.
 8. **Verifying the correctness of software and hardware systems:** Finite automata can be used to verify the correctness of software and hardware systems by checking whether they satisfy certain requirements or specifications.
 9. **Developing language models and machine learning algorithms:** Finite automata are used in the development of language models and machine learning algorithms, which are used in natural language processing and other applications. They can help identify patterns in data and make predictions based on those patterns.
 10. **Studying formal languages and their properties:** Finite automata are used in the study of formal languages and their properties. They can help identify the structure and rules of different types of languages and the limitations of those languages in terms of expressiveness and complexity.
2. **Push Down Automata** - A pushdown automaton (PDA) is a mathematical model similar to a finite automaton, but with a stack for memory. It has states, input symbols, a transition function that maps states, input symbols, and top stack symbols to other states and stack operations, and a set of accepting states. The PDA reads input symbols and manipulates the stack, either accepting or rejecting the input. PDAs are used to model more complex systems in computer science and linguistics. Some applications of push down automata are as follows:
1. **Parsing context-free grammars:** Pushdown automata are often used to parse context-free grammars. Context-free grammars are a type of formal grammar that is commonly used to describe the structure of programming languages, markup languages, and other languages.
 2. **Implementing stack-based memory management:** Pushdown automata can be used to implement stack-based memory management in programming languages. In this type of memory management, memory is allocated and deallocated from a stack data structure. Pushdown automata can be used to implement the stack and manage the allocation and deallocation of memory.
 3. **Modeling natural language processing:** Pushdown automata can be used to model natural language processing (NLP) systems. NLP is a subfield of artificial intelligence that focuses on the interaction between computers and human languages. Pushdown automata can be used to recognize and parse sentences in natural language, which can be used to build more advanced NLP systems.
 4. **Recognizing context-sensitive languages:** Pushdown automata can be used to recognize context-sensitive languages, which are a type of formal language that is more complex than context-free languages. Context-sensitive languages are used in many applications, such as natural language processing and database management.
 5. **Modeling network protocols:** Pushdown automata can be used to model network protocols, such as the Transmission Control Protocol (TCP) and the User Datagram Protocol (UDP). These protocols are used to facilitate communication between computers over a network. Pushdown automata can be used to recognize and process the messages exchanged between computers.

3. **Turing Machine** - A Turing machine is a mathematical model of a general-purpose computer that can perform any computation that any other computer can. Here are some applications of Turing machines:
1. **Artificial intelligence:** Turing machines can be used in artificial intelligence applications, such as natural language processing and machine learning. Turing machines can be used to process and analyze large amounts of data, recognize patterns, and learn from examples.
 2. **Cryptography:** Turing machines can be used in cryptography, which is the study of techniques for secure communication. Turing machines can be used to encrypt and decrypt messages, and to generate random numbers.
 3. **Algorithmic complexity theory:** Turing machines are used in algorithmic complexity theory, which is the study of the computational resources required to solve problems. Turing machines can be used to analyze the time and space complexity of algorithms, and to determine whether a problem is solvable by a computer.
 4. **Compiler design:** Turing machines can be used in compiler design, which is the process of transforming source code into executable code. Turing machines can be used to implement the parsing and translation phases of the compiler, which involves recognizing the syntax of the source code and generating the corresponding executable code.
 5. **Computation theory:** Turing machines are used in computation theory, which is the study of the nature and limits of computation. Turing machines can be used to define the class of computable functions, which are functions that can be computed by a Turing machine.

Q. 5) A

Q.5) A Moore m/c that converts all occurrences of 100 to 101 in string over $\{0,1\}$.
convert it to Mealy m/c.

Moore m/c



Moore to Mealy

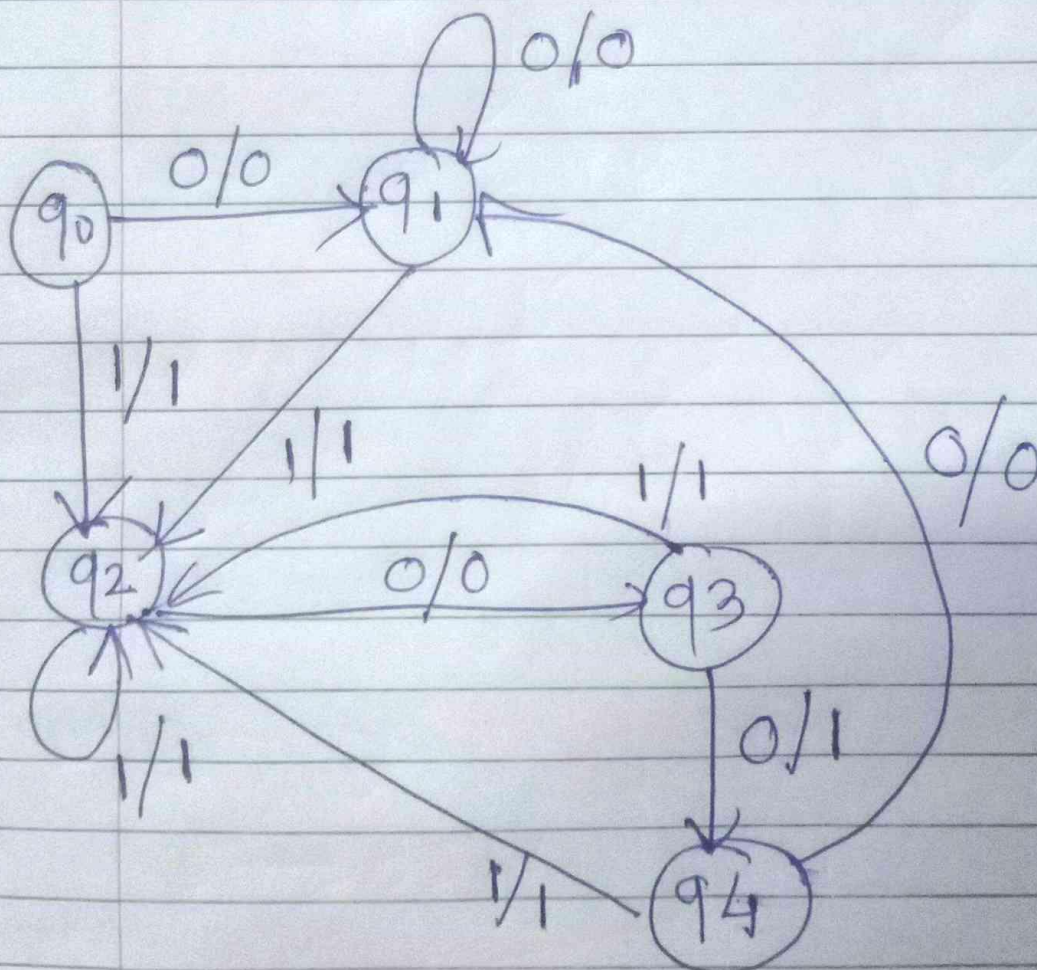
Transition table of Moore

	0	1	Δ
q ₀	q ₁	q ₂	-
q ₁	q ₁	q ₂	0
q ₂	q ₃	q ₂	1
q ₃	q ₄	q ₂	0
q ₄	q ₁	q ₂	1

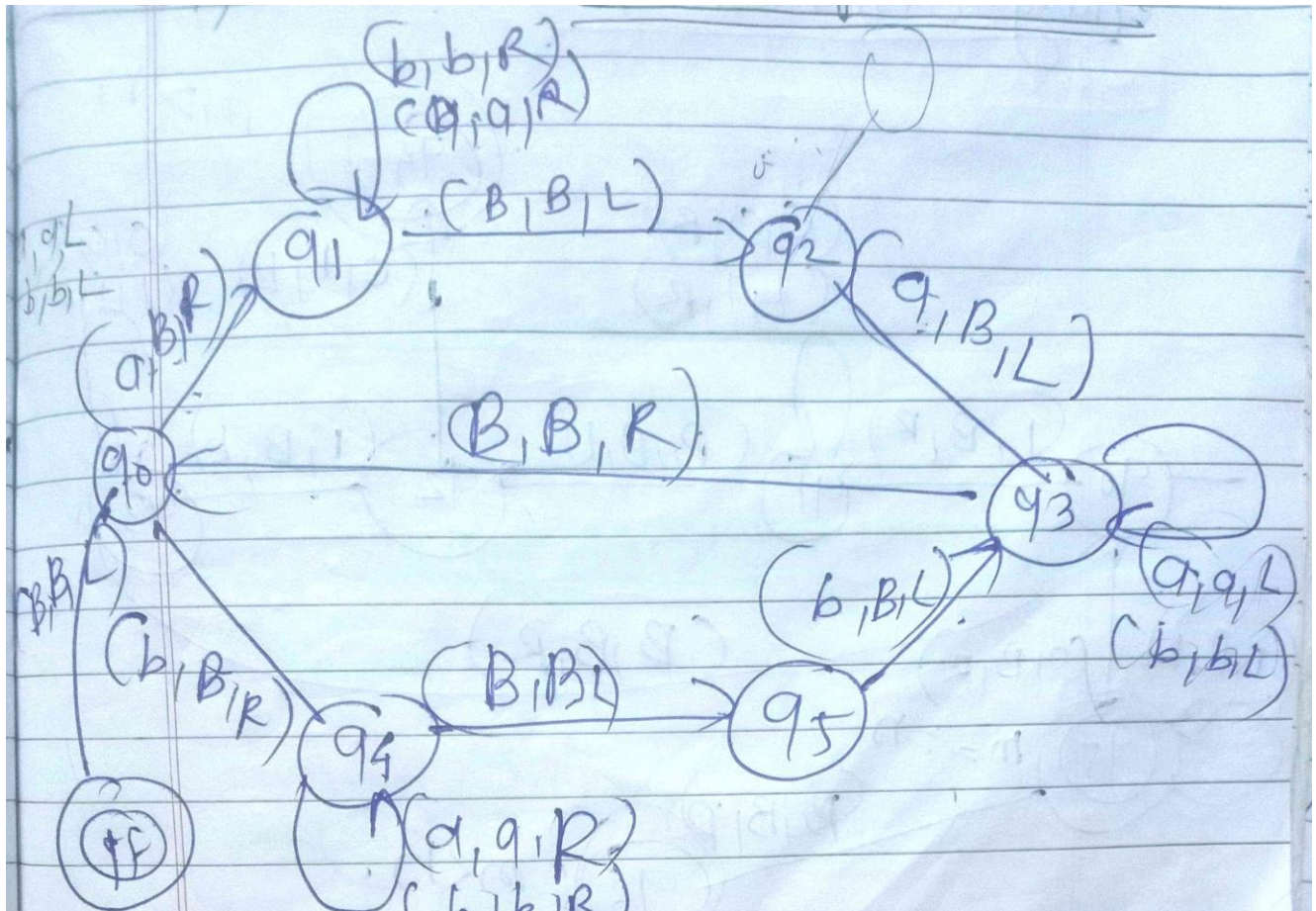
Transition table of Melay

	0	1
q_0	$(q_1, 0)$	$(q_2, 1)$
q_1	$(q_1, 0)$	$(q_2, 1)$
q_2	$(q_3, 0)$	$(q_2, 1)$
q_3	$(q_4, 1)$	$(q_2, 1)$
q_4	$(q_1, 0)$	$(q_2, 1)$

Melay m/c will be as following.



Q. 5) B Turing Machine to accept all palindromes.



Q. 6) A

Decision Properties

1. Emptiness and Non emptiness
2. Finiteness and Infiniteness
3. Membership
4. Equivalence

Emptiness and Non Emptiness

To check whether a given FA accepts empty language or non-empty language.

Steps to check emptiness:

- 1) Select the state that cannot be reached from the initial states & delete them (remove unreachable states).

2) If the resulting machine contains at least one final states, so then the finite automata accepts the non-empty language.

3) If the resulting machine is free from final state, then finite automata accepts empty language.

Finiteness and Infiniteness

It states that the language accepted by a given FA is finite or not.

Steps:

- 1) Select the state that cannot be reached from the initial state & delete them (remove unreachable states).

2) Select the state from which we cannot reach the final state & delete them (remove dead states).

3) If the resulting machine contains loops or cycles then the finite automata accepts infinite language.

4) If the resulting machine do not contain loops or cycles then the finite automata accepts infinite language.

Membership

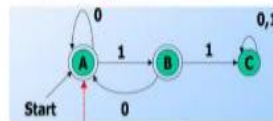
Membership is a property to verify an arbitrary string is accepted by a finite automata or not.

It is decidable, because by reading the string if it reaches the final state, then it is a member.

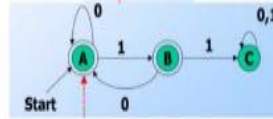
Otherwise, if the string halts in a non-final state then it is said to be not a member of the FA.

Is string w accepted by given FA? $w=01011$.

Step I: 0 1 0 1 1



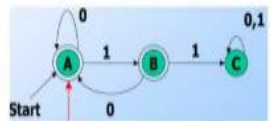
Step II: 0 1 0 1 1



Step III: 0 1 0 1 1



Step IV: 0 1 0 1 1



Step V: 0 1 0 1 1



Step VI: 0 1 0 1 1



Equivalence

Steps to identify equivalence:

1. If initial state is final state for one automaton, then in second automaton also initial state must be final state for them to be equivalent.
2. For any pair of states $\{q_i, q_j\}$ the transition for the input $a \in \Sigma$ is defined by $\{q_a, q_b\}$ where $\delta\{q_i, a\} = q_a$ and $\delta\{q_j, a\} = q_b$. The two automata are not equivalent if for a pair $\{q_a, q_b\}$ one is intermediate state and other is final state.

The Post Correspondence Problem (PCP)

is a classic problem in theoretical computer science and formal languages? It was introduced by Emil Post in 1946 and is known for its undecidability. Here are some notes on the Post Correspondence Problem:

- Definition:

- Given a set of dominos, each containing a string on its top and bottom, the goal of the Post Correspondence Problem is to determine if there exists a sequence of dominos that can be stacked (concatenated) together such that the resulting string on the top matches the string on the bottom.

- Formally, given a finite set of dominos $D = \{(a_1, b_1), (a_2, b_2), \dots, (a_n, b_n)\}$, where a_i and b_i are non-empty strings, the problem is to find a sequence of indices $i_1, i_2, \dots, i_k$ such that $a[i_1]a[i_2]\dots a[i_k] = b[i_1]b[i_2]\dots b[i_k]$.

- Example:

- Let's consider a set of dominos $D = \{(ab, ba), (a, ab), (b, bb)\}$.

- We want to determine if there exists a sequence of dominos that can be stacked together to form the same string on the top and bottom.

- By trying different combinations, we can find a solution: $(ab, ba), (a, ab), (a, ab)$.

- When we stack these dominos, we get "ababa" on the top and "baabab" on the bottom, which matches.

- Undecidability:

- The Post Correspondence Problem is undecidable, meaning there is no algorithm that can solve it for all possible inputs.

- The undecidability of the problem was proved by showing its reduction to the Halting Problem, which is known to be undecidable.

- Applications:

- The Post Correspondence Problem has significance in the study of formal languages, computability theory, and algorithmic complexity.

- It serves as a tool for proving the undecidability of other problems by reduction.

- The problem also has connections to other areas, such as DNA computing and self-replicating systems.

- Complexity:

- The Post Correspondence Problem is NP-hard, which means that any problem in the complexity class NP can be reduced to it.

- Although the problem is undecidable, it can be solved for specific instances by exhaustive search or using specific techniques.

C) Variants of Turing Machines

Turing Machine Variants:

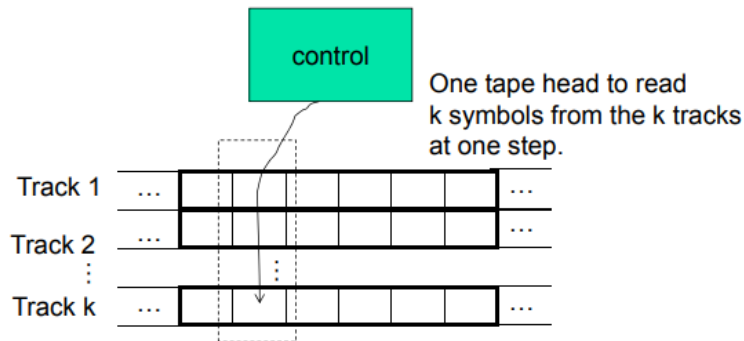
Multi tape TM:

Multi Track TM:

A multi-track Turing machine is a variant of the simple Turing machine. It consists of multiple tapes with a single head pointer. They are beneficial in solving complex problems and limit the amount of work, unlike a simple Turing machine.

As there is a single head, the direction of all the tapes changes together. The input is placed in the first tape and is transferred to the other tapes as per convenience.

■ TM with multiple tracks, but just one unified tape head

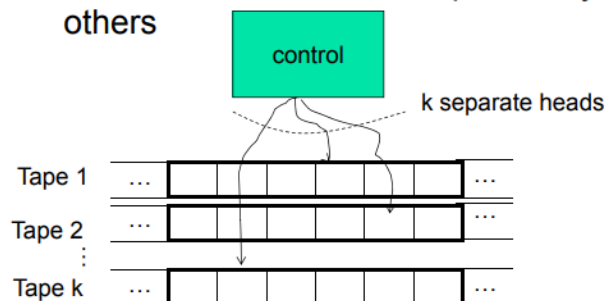


Multi tape Turing Machine

Multi-tape Turing Machines have multiple tapes where each tape is accessed with a separate head. Each head can move independently of the other heads. Initially the input is on tape 1 and others are blank. At first, the first tape is occupied by the input and the other tapes are kept blank. Next, the machine reads consecutive symbols under its heads and the TM prints a symbol on each tape and moves its heads.

■ TM with multiple tapes, *each tape with a separate head*

- Each head can move independently of the others



Non Deterministic Turing Machine

It is similar to DTM except that for any input and current state it has a number of choices.

A string is accepted by a NDTM if there is a sequence of moves that leads to a final state

Multi-head Turing machine

It has a number of heads instead of one.

Each head independently reads/ writes symbols and moves left/right or keeps stationery.

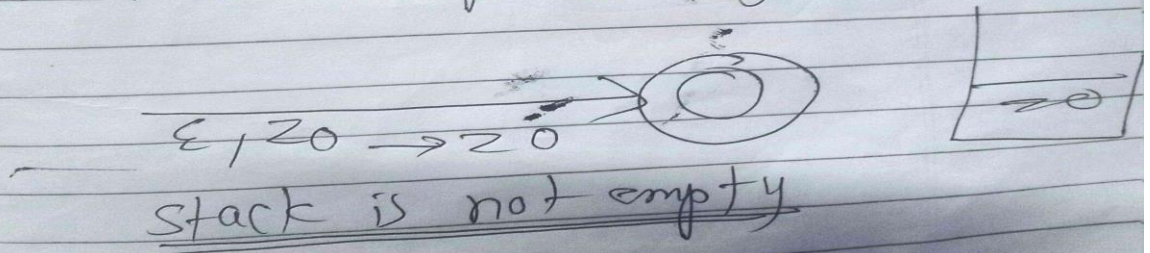
Off-line Turing Machine

An offline Turing machine has two tapes, which are as follows –

Acceptance by PDA

1) Acceptance by final state

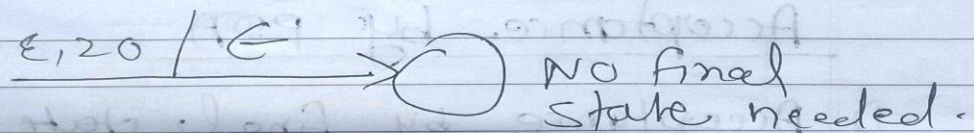
After reading complete i/p string if the PDA reaches the final state then it is acceptance by final state.



2) Acceptance by empty stack.

After reading complete i/p string if the stack is empty, then the i/p string is accepted by PDA.

Stack is $a^k b^k \epsilon$. push both a on stack and on any occurrence of b pop one a. At the end stack is having z_0 , and string is complete with ϵ and for ϵ if you are also popping z_0 then it is acceptance by empty stack.

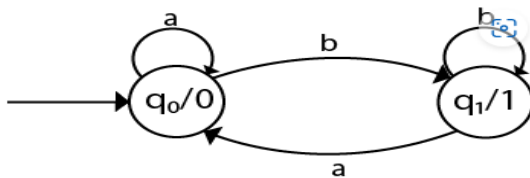


E)

Conversion from Moore machine to Mealy Machine

In the Moore machine, the output is associated with every state, and in the mealy machine, the output is given along the edge with input symbol. The equivalence of the Moore machine and Mealy machine means both the machines generate the same output string for same input string.

We cannot directly convert Moore machine to its equivalent Mealy machine because the length of the Moore machine is one longer than the Mealy machine for the given input. To convert Moore machine to Mealy machine, state output symbols are distributed into input symbol paths. We are going to use the following method to convert the Moore machine to Mealy machine.

**Solution:**

The transition table of given Moore machine is as follows:

Q	a	b	Output(λ)
q0	q0	q1	0
q1	q0	q1	1

$$\begin{aligned}
 \lambda'(q_0, a) &= \lambda(\delta(q_0, a)) \\
 &= \lambda(q_0) \\
 &= 0
 \end{aligned}$$

$$\begin{aligned}
 \lambda'(q_0, b) &= \lambda(\delta(q_0, b)) \\
 &= \lambda(q_1) \\
 &= 1
 \end{aligned}$$

State q0

$$\begin{aligned}
 \lambda'(q_1, a) &= \lambda(\delta(q_1, a)) \\
 &= \lambda(q_0) \\
 &= 0
 \end{aligned}$$

$$\begin{aligned}
 \lambda'(q_1, b) &= \lambda(\delta(q_1, b)) \\
 &= \lambda(q_1) \\
 &= 1
 \end{aligned}$$

State q1

Hence the transition table for the Mealy machine can be drawn as follows:

Q \ Σ	Input 0		Input 1	
	State	O/P	State	O/P
q ₀	q ₀	0	q ₁	1
q ₁	q ₀	0	q ₁	1

